



CSCI/ECEN 3302

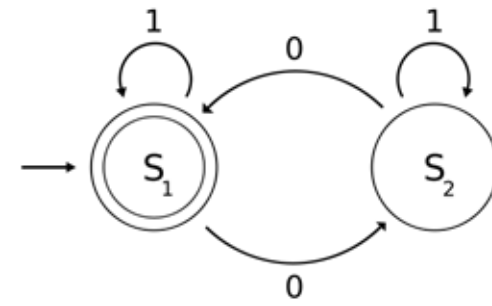
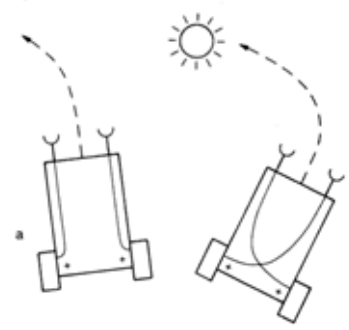
Introduction to Robotics

Alessandro Roncone

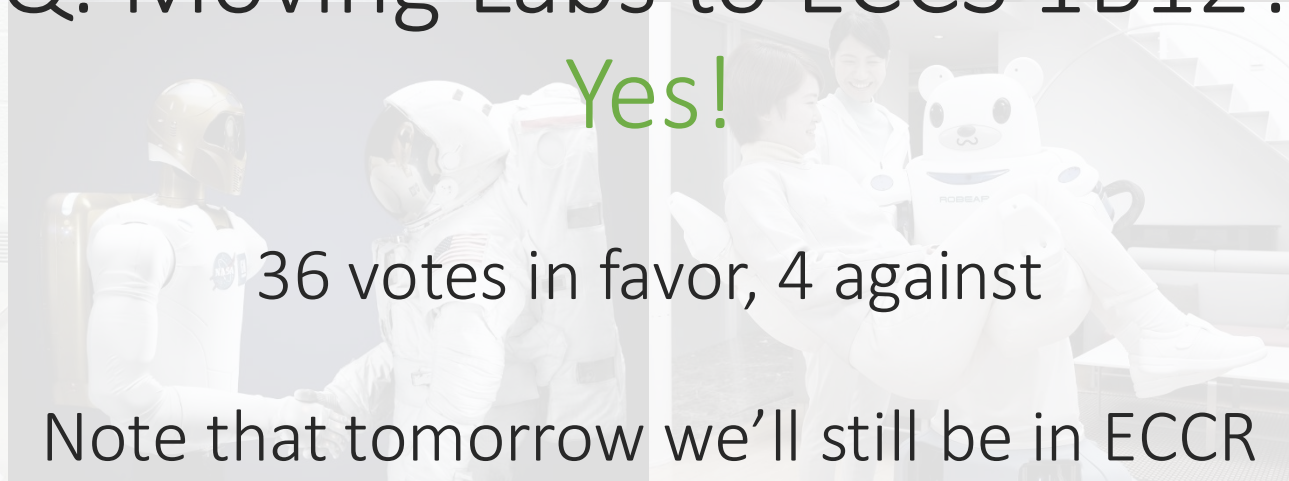
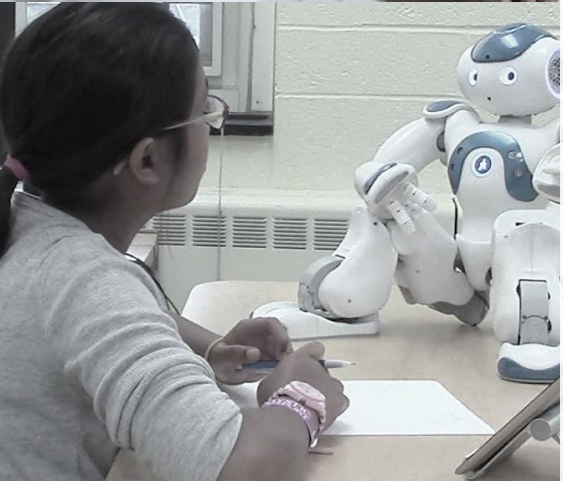
aroncone@colorado.edu

<https://hiro-group.ronc.one>

Administrivia



- **HW0** – grades are out!
 - Please reach out (Piazza, email) if you have any questions
- **Lab1** is due tonight!
 - I am expecting all groups to be finalized by the start of Lab2, but there may be some additional minor reshuffling.
 - Submissions are group submissions – one submission per team!
- **HW1** is out! Due next Tuesday 2/3
- **Lab2** will be out tomorrow! Due in two weeks on Tuesday 2/11
 - Remember: Labs will get harder and harder -> you are not expected to carry the workload of your entire team!
 - Please reach out if you have issues with your teammates



Q. Moving Labs to ECCS 1B12?

Yes!

36 votes in favor, 4 against

Note that tomorrow we'll still be in ECCR
1B40!



Why Humanoids?

The case **for** humanoids

Universal Interface

- ✓ **Brownfield Adaptation:** Stairs, door handles, and tools are designed for human hands. A humanoid form bypasses the need to retroactively "fix" infrastructure for robots.

The Data Flywheel

- ✓ **Imitation Learning:** We have petabytes of human video data. A humanoid morphology allows for direct, 1:1 mapping of human motion to robot policy (Sim2Real), drastically reducing training time.

Social Fluency

- ✓ **Collaborative Safety:** Humanoids communicate intent through familiar cues—gaze, head orientation, and posture—making them safer coworkers in shared spaces.



Why Humanoids?

The case **against** humanoids

While the "human form" is ideal for the environment, it is often suboptimal for engineering!!

The Complexity Penalty

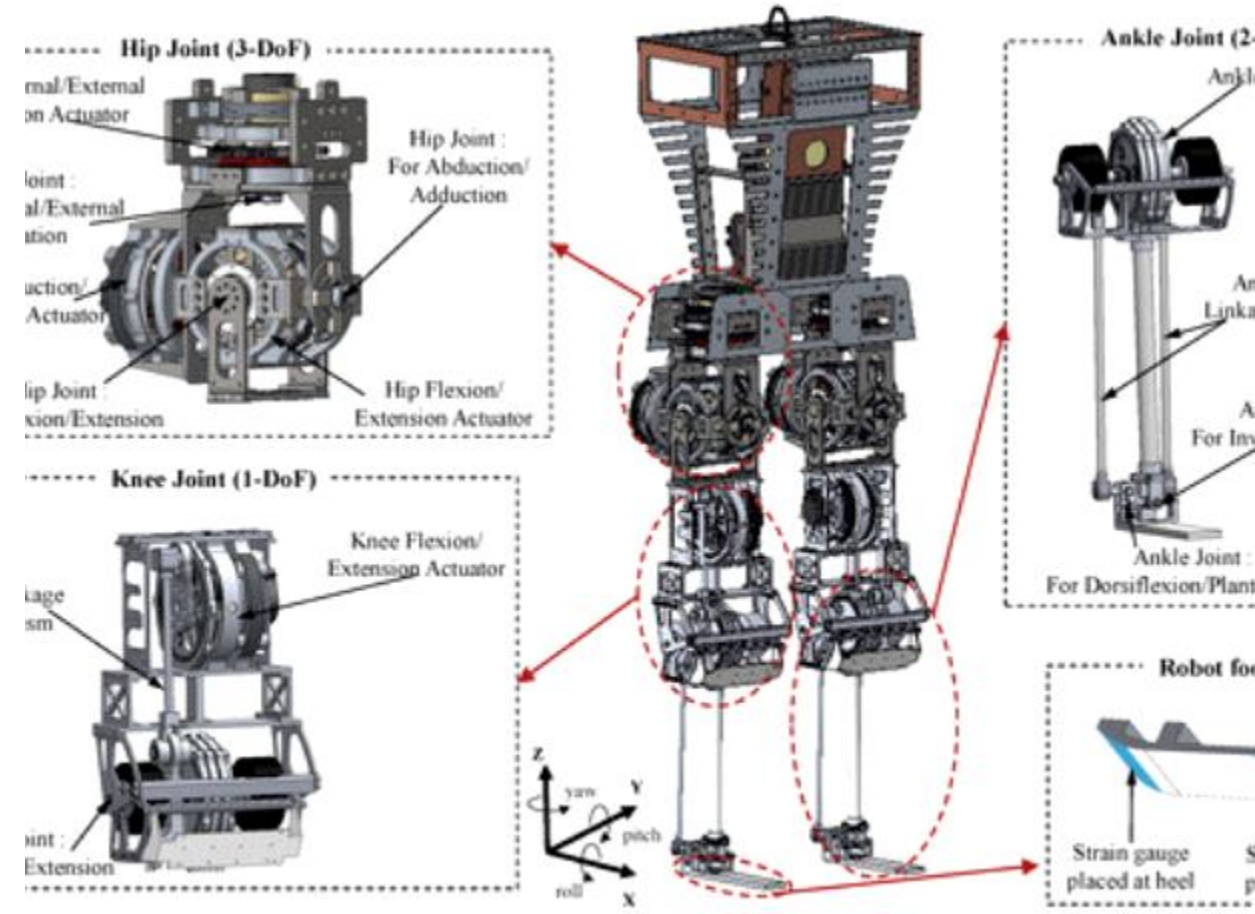
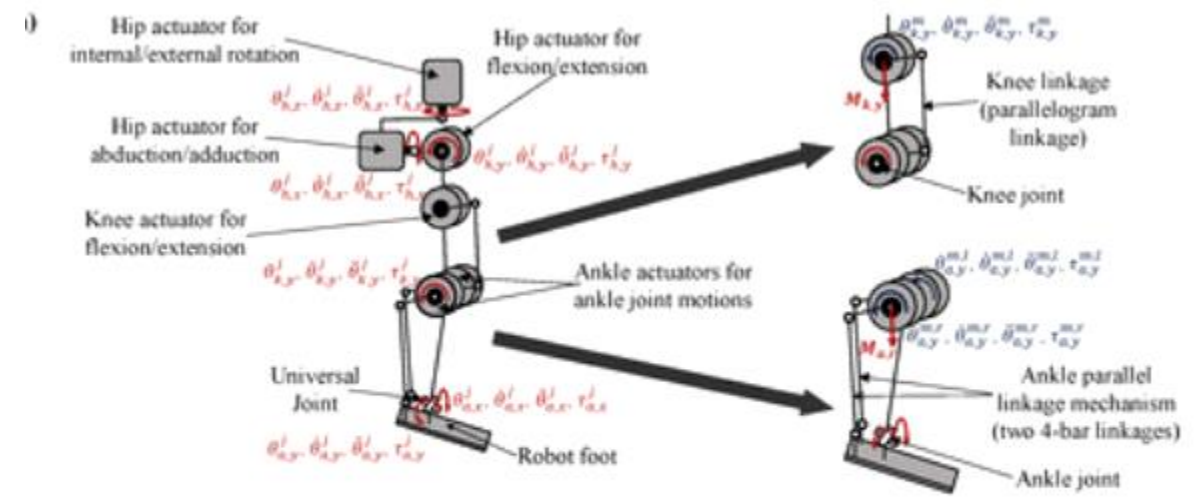
High BOM Cost: Bipedal locomotion requires 20+ actuators vs. 2 for a wheeled robot. High Degrees of Freedom (DoF) exponentially increase points of failure and maintenance costs!!

The Safety Paradox

Stability Risk: Wheels don't fall; legs do. A 150lb humanoid losing balance is a heavy projectile. In industrial settings, the energy required to simply "stand still" is a significant inefficiency.

Specialization Gap

Jack of all trades: For specific tasks like moving pallets, a forklift (AMR) is faster, stronger, and 10x more energy-efficient than a humanoid.

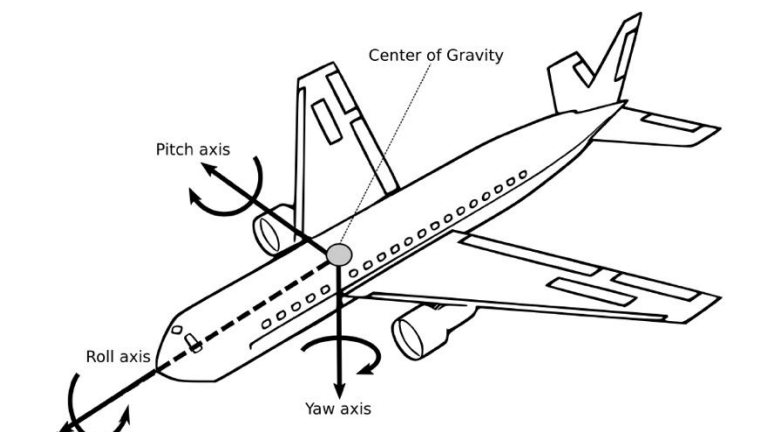
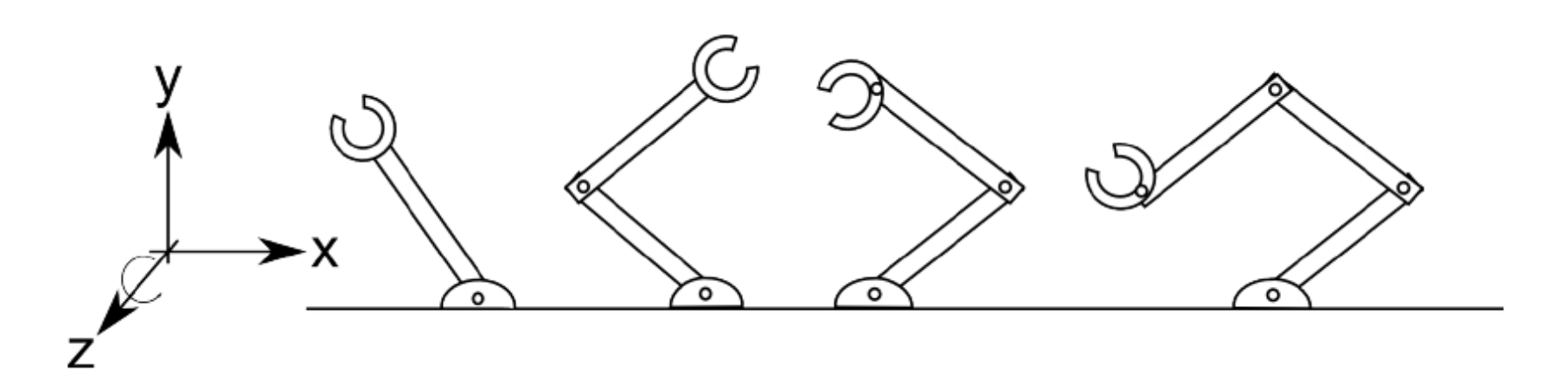


Chapter 2 in the book



RECAP

Locomotion

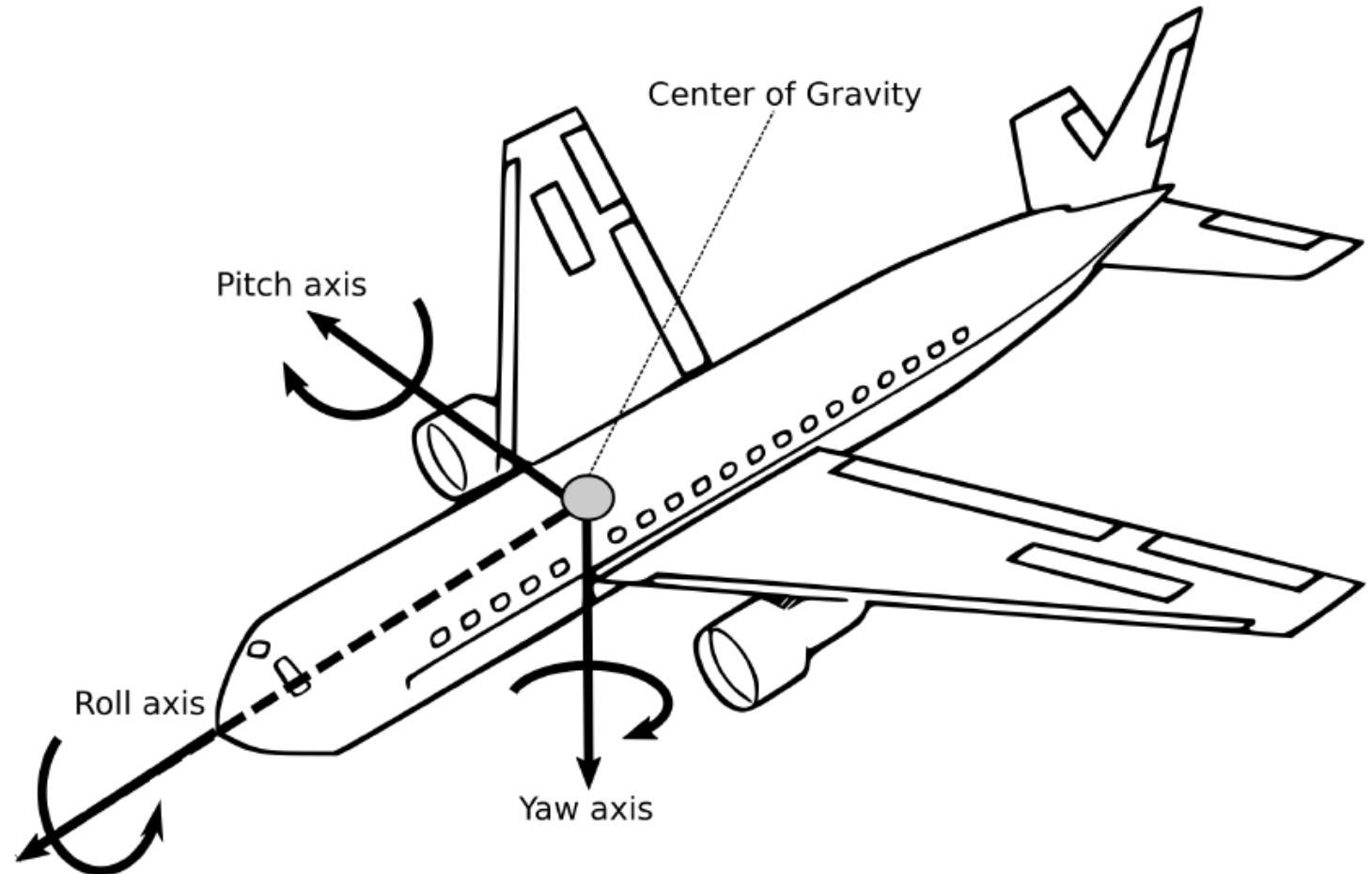


Actuation

- **Locomotion** – Moving oneself
 - Enacting a force on / utilizing a force within the environment
 - Rolling, walking, running, jumping, sliding, flying, etc.
- **Manipulation** – Moving others
 - Enacting forces on objects

Degrees of Freedom [DoFs]

- 6-dimensional:
 - X, Y, Z
 - Yaw, Pitch, and Roll
- 3D position + 3D orientation = 6D pose
- Fully characterizes the state of the plane



Degrees of Freedom in Joint Space

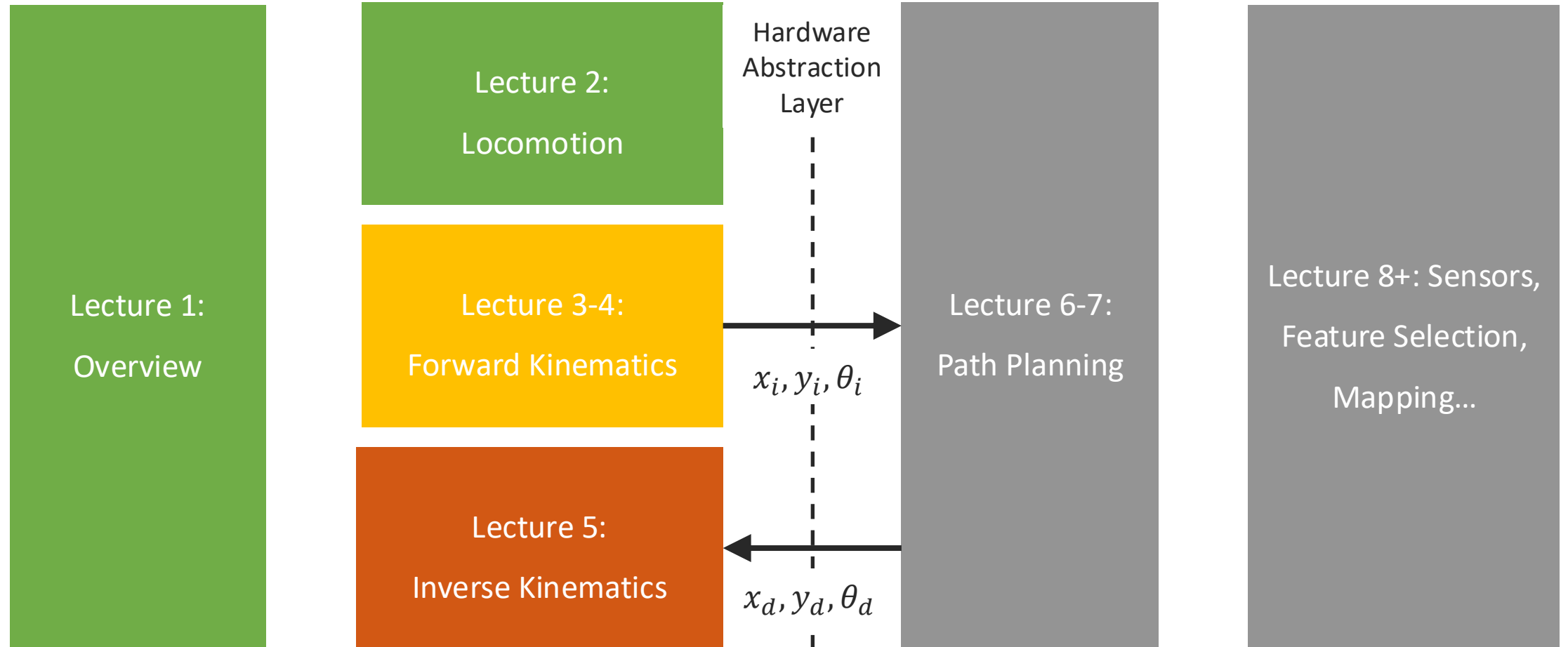
- DoF = Number of **controllable points of actuation**
- Most robot arms have 6-DoF at their end effector
 - DoF in **cartesian space**
 - They are not directly controllable
- Robots can be controlled through motors in the joints
 - DoFs in **joint space**
 - **REDUNDANCY**: Robot DoF > Environment DoF
 - Why would you want this?



cartesian space == operational space ==
task space == environment

How many DoFs does a
human arm have?

Roadmap



Chapter 3 in the
book

Module 1 – MECHANISMS

Part II – Forward Kinematics

Kinematics

- **Kinematics** – Geometry of Motion
 - The way parts of a robot move with respect to each other and the environment
 - Position (x)
 - Speed (x')
- **Dynamics** – Newtonian Mechanics
 - Acceleration (x'')
 - Jerk (x''')
- Trivia: other derivatives of position?

Abstractions of
increasing complexity:

Kinematics



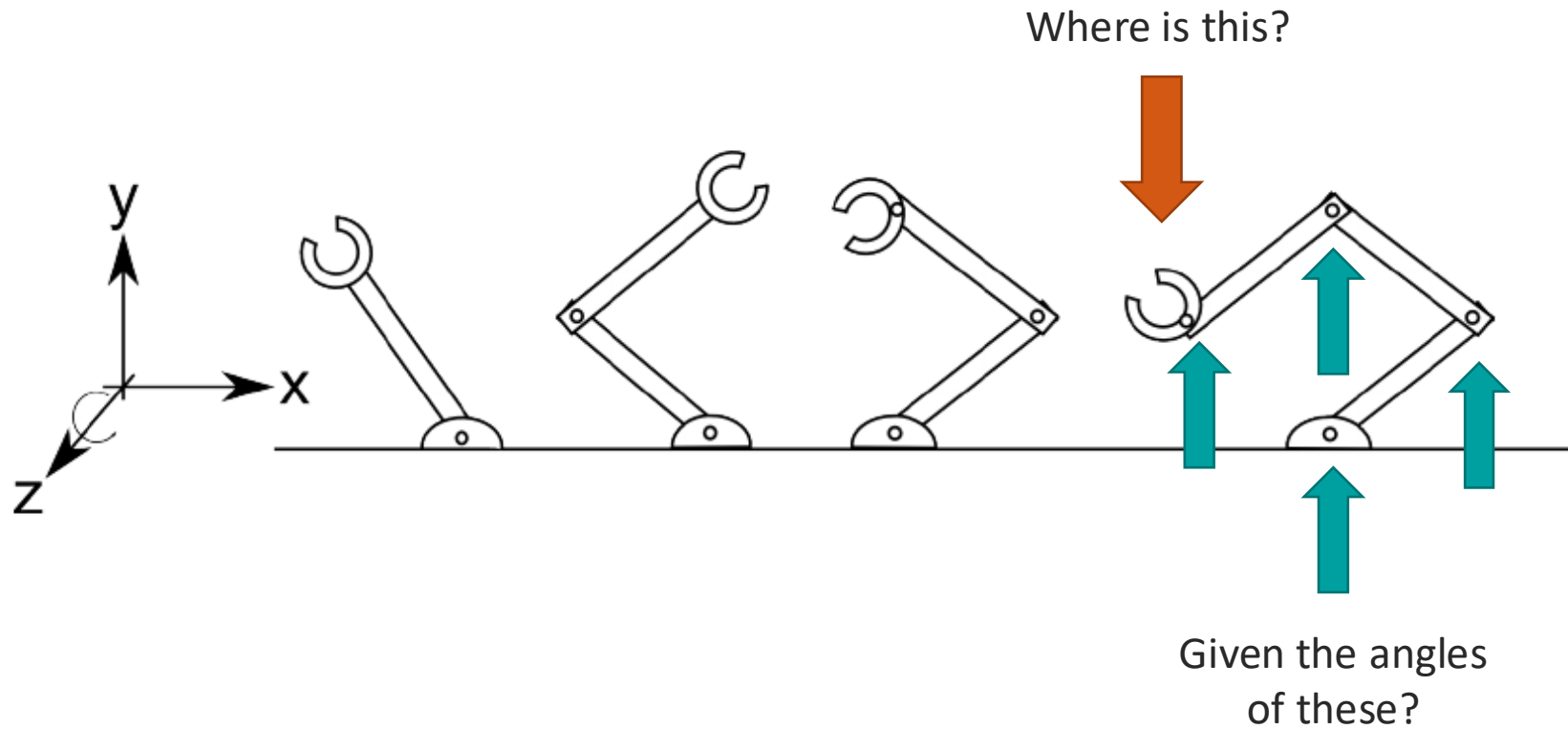
Statics



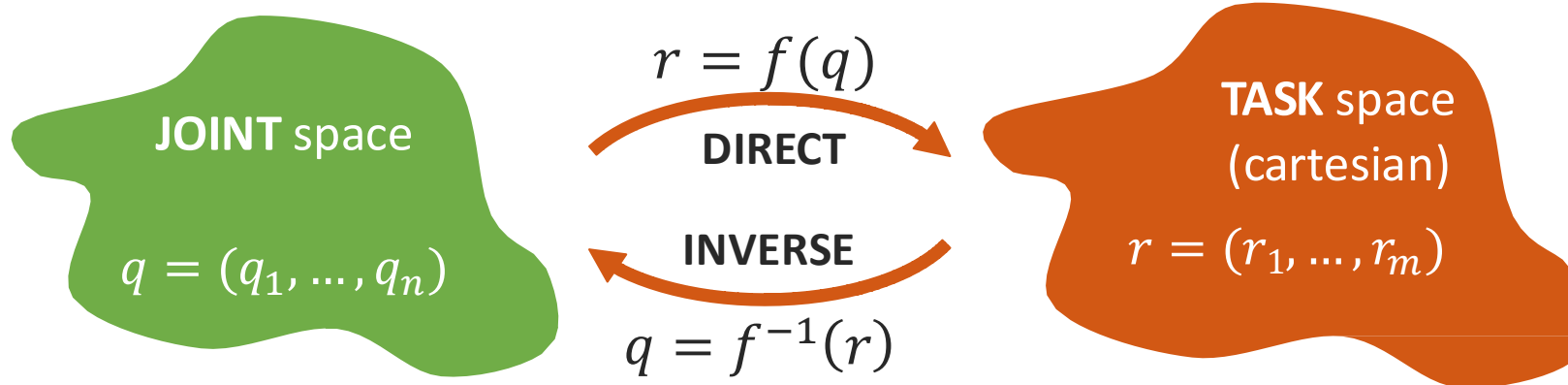
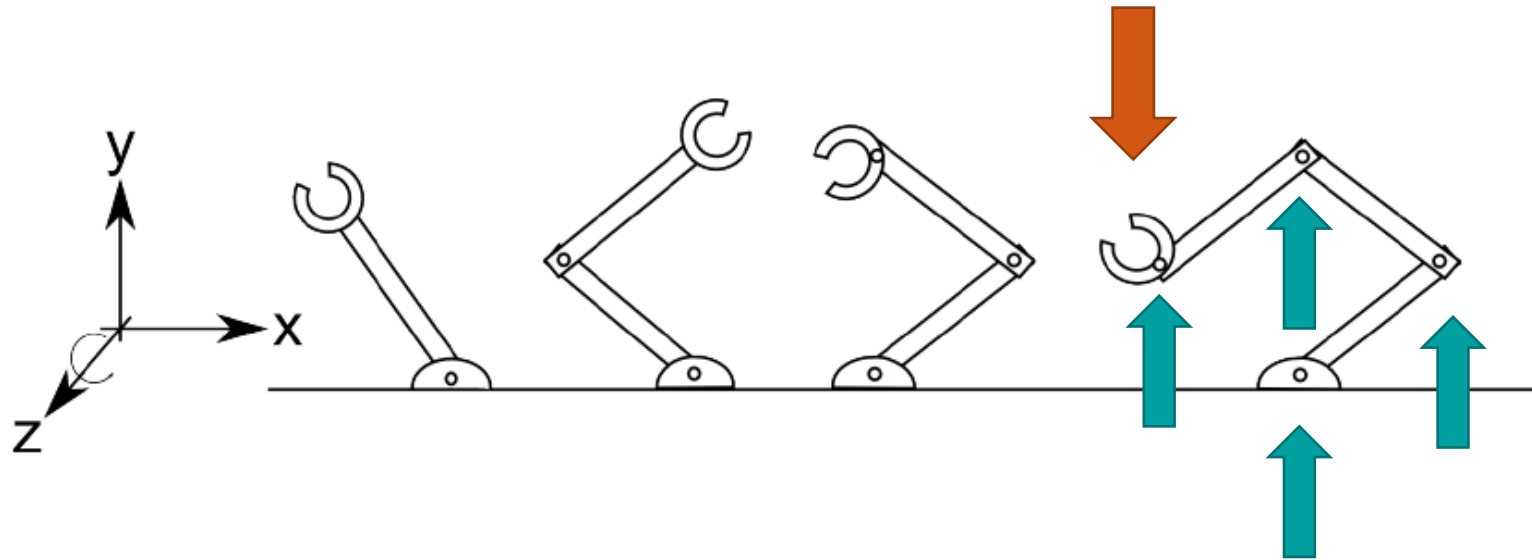
Dynamics



Forward (Direct) Kinematics



Forward (Direct) Kinematics



Forward Kinematics

Given a robot configuration, what is the end-effector's pose?

Inverse Kinematics

How to move an arm in space in order to reach for a specific pose [or position, or orientation]?



Forward Kinematics – Definitions

- Coordinate Systems
- Nested Coordinate Systems
- Introduction to Rotations: Deriving 2D Rotation Equations

Rigid Body Transformations

- Rigid Body:

- $O \in \mathbb{R}^3$
- $\forall p, q \in O$:
- $\|p(t) - q(t)\| = \|p(0) - q(0)\|$

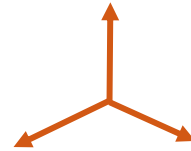
- Rigid Body Transformation:

- A mapping $\mathbf{g}: \mathbb{R}^3 \rightarrow \mathbb{R}^3$ is a rigid body transformation if:
- Length is preserved:

$$\|g(p) - g(q)\| = \|p - q\| \quad \forall p, q \in \mathbb{R}^3$$

- Cross product is preserved:

$$g_*(v \times w) = g_*(v) \times g_*(w) \quad \forall v, w \in \mathbb{R}^3$$



Specifying Coordinate Systems

Points

- Express locations

Vectors

- Displacement, movement, force

Normals

- Are unit length
- Useful for orientations

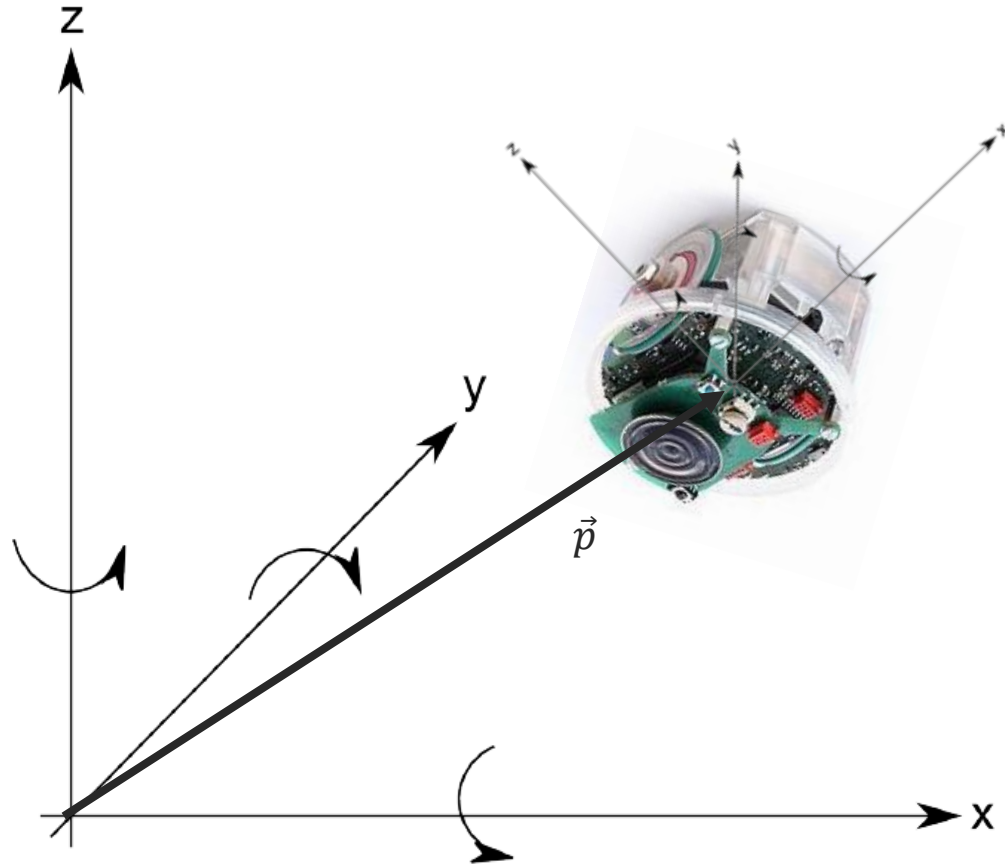
Coordinates

- Numerical representation of the above
- Requires a **coordinate system**

Coordinate System

- Composed of three **orthonormal** vectors of unit length

What do we need to describe pose?



Notation

- $\vec{v} = c_1 \vec{b}_1 + c_2 \vec{b}_2 + c_3 \vec{b}_3$

- **Basis** vectors $\vec{b}$ identify the coordinate system axes

- **Coefficients** c express how far to travel along each axis

- $\vec{v} = [\vec{b}_1 \quad \vec{b}_2 \quad \vec{b}_3] \begin{bmatrix} c_1 \\ c_2 \\ c_3 \end{bmatrix}$

Shorthand: $\vec{v} = \vec{b}^T c$

Example – no rotation, only translation

$$\blacksquare \vec{p} = c_1 \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix} + c_2 \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix} + c_3 \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}$$

- Basis vectors $\vec{b}$ identify the coordinate system axes
- Basis vectors need to be *orthonormal*
- Coefficients c express how far to travel along each axis

$$\blacksquare \vec{p} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} c_1 \\ c_2 \\ c_3 \end{bmatrix}$$

Shorthand: $\vec{p} = R\vec{c} = \vec{c}$

Coordinate System

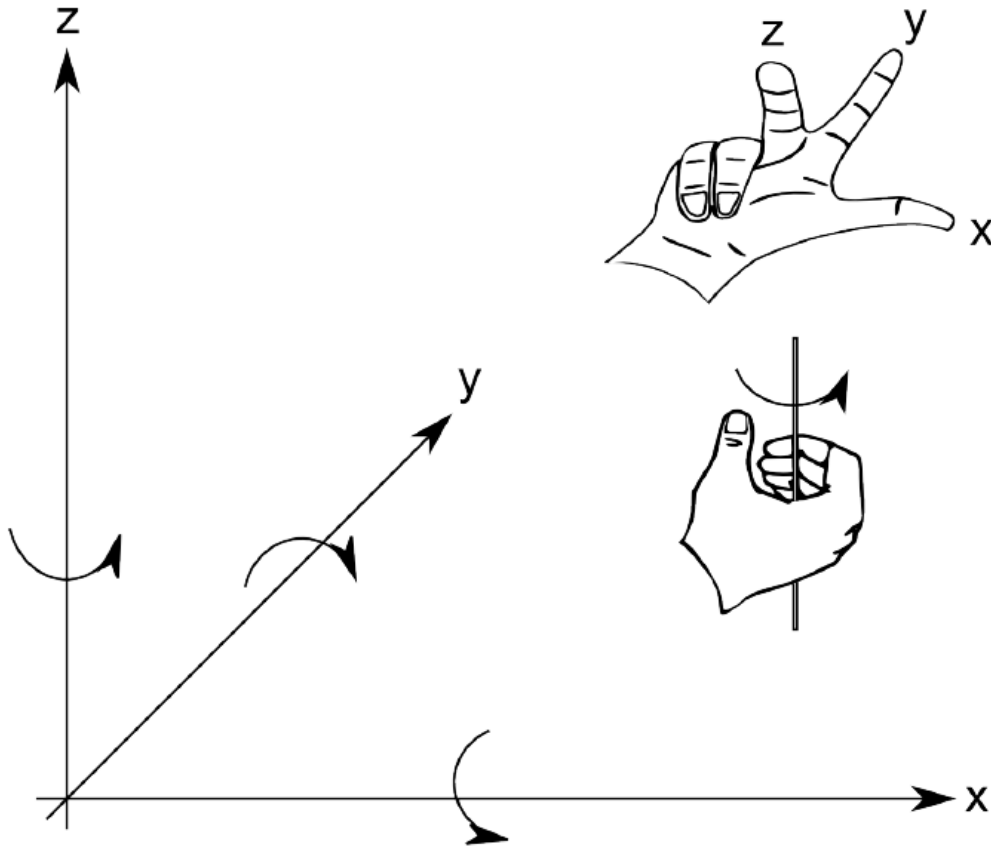
- Requires a point origin $\tilde{o}$ and a set of basis vectors $\vec{b}$
- Expressing a point in a coordinate system:

$$\hat{S} = \{\tilde{o}, \vec{b}\}$$

$$\hat{s}_p = \tilde{o} + \sum_i c_i \vec{b}_i$$

$$\hat{s}_p = [\vec{b}_1 \quad \vec{b}_2 \quad \vec{b}_3 \quad \tilde{o}] \begin{bmatrix} c_1 \\ c_2 \\ c_3 \\ 1 \end{bmatrix}$$

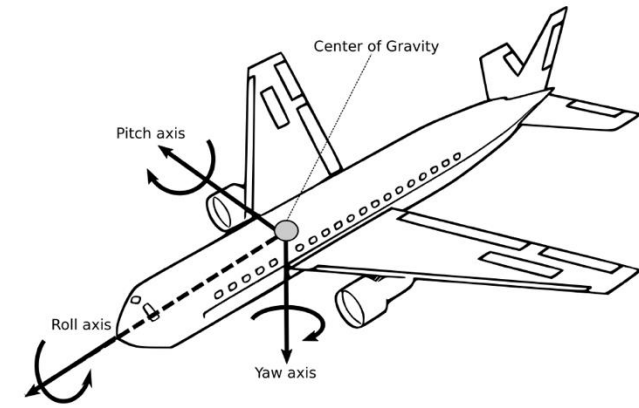
Coordinate Systems and Right Hand Rule



- Thumb along x-axis
- Index along y-axis
- Middle along z-axis
- It's the order that counts!

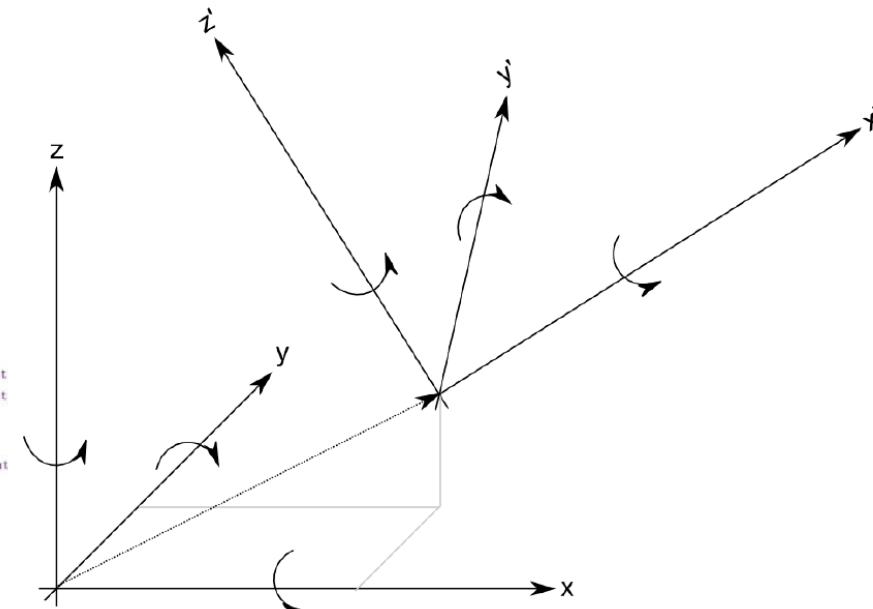
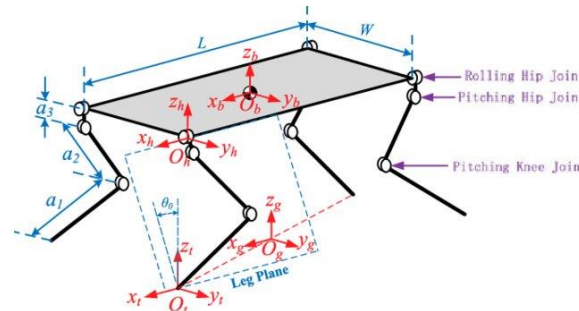
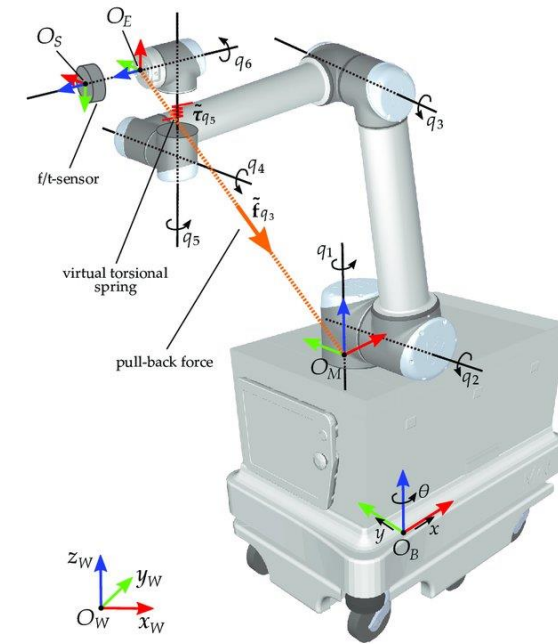
Nested Coordinate Systems

- Recall: an airplane can be modeled as a single rigid body
 - A single coordinate system (i.e. a single pose) **fully characterizes its kinematic state**
- How can a robot manipulator be modeled?
 - **Multiple rigid bodies** connected together by joints
 - Each DoF / point of actuation presents a **new Coordinate System** w.r.t. the previous linkage



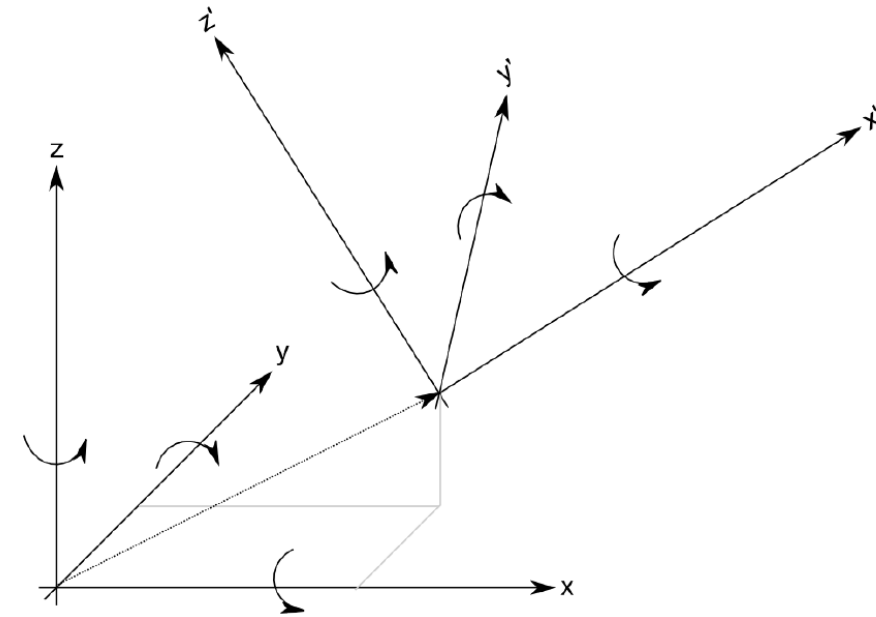
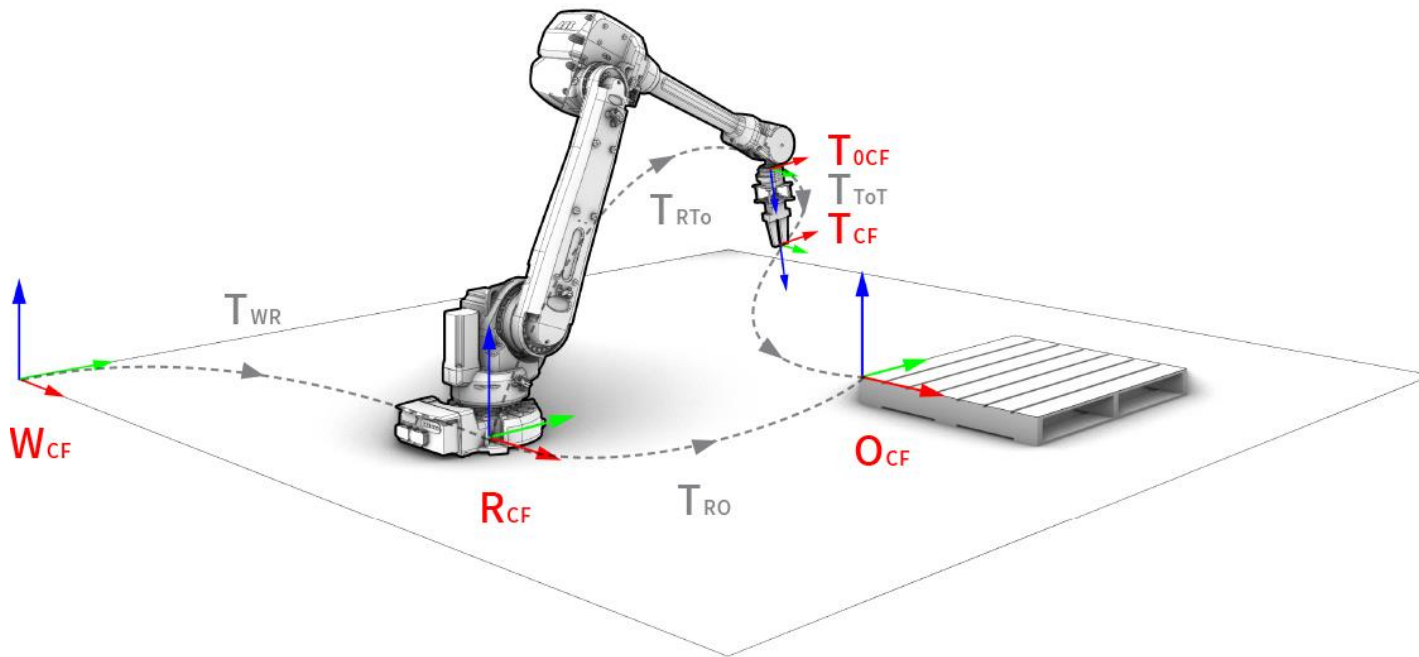
Nested Coordinate Systems

- Each DoF / point of actuation presents a **new Coordinate System with respect to the previous linkage!!**
- Applies to both manipulators and mobile platforms!



Nested Coordinate Systems

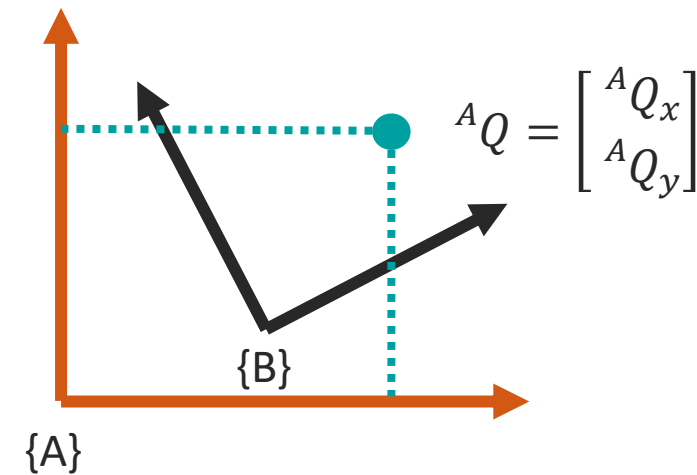
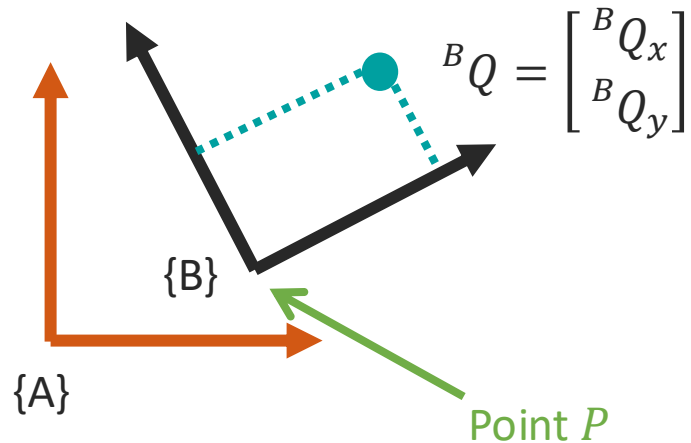
- This idea can be used beyond the robot as well!!



- What other domains **beyond robotics** this is used in?

Nested Coordinate Systems

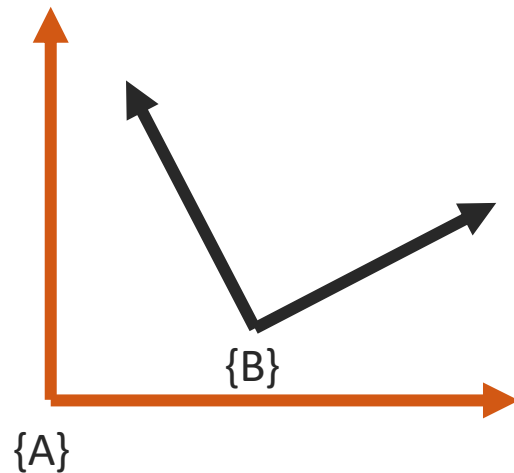
How can we express point Q , with position known in coordinate system $\{B\}$ centered at P , in coordinate system $\{A\}$ with origin $(0,0)$?



Nested Coordinate Systems

How can we express point Q , with position known in coordinate system $\{B\}$ centered at P , in coordinate system $\{A\}$ with origin $(0,0)$?

- How can we convert between $\{A\}$ and $\{B\}$?

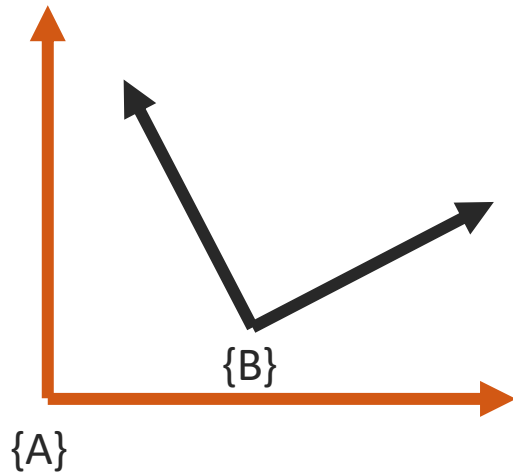


We need to find the **translation** and **rotation** that would move the axes of $\{A\}$ onto $\{B\}$

Nested Coordinate Systems

How can we convert between {A} and {B}?

We need to find the **translation** and **rotation** that would move the axes of {A} onto {B}



Translation:

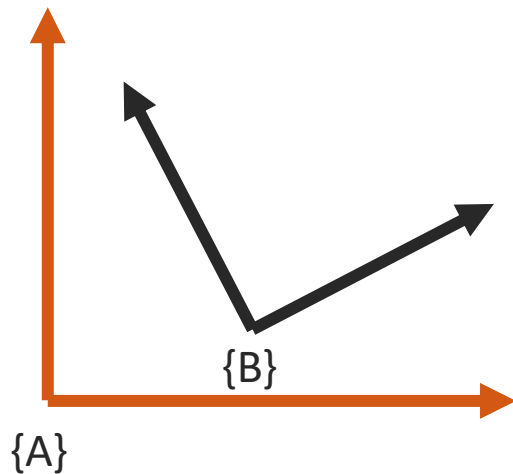
Subtract {A} from {B} to get the difference between origins.

$$\begin{aligned}T_x &= \hat{B}_x - \hat{A}_x \\T_y &= \hat{B}_y - \hat{A}_y \\T_z &= \hat{B}_z - \hat{A}_z\end{aligned}$$

Nested Coordinate Systems

How can we convert between $\{A\}$ and $\{B\}$?

We need to find the **translation** and **rotation** that would move the axes of $\{A\}$ onto $\{B\}$



Rotation:

For each axis of $\{B\}$, we need to find out how it maps onto $\{A\}$

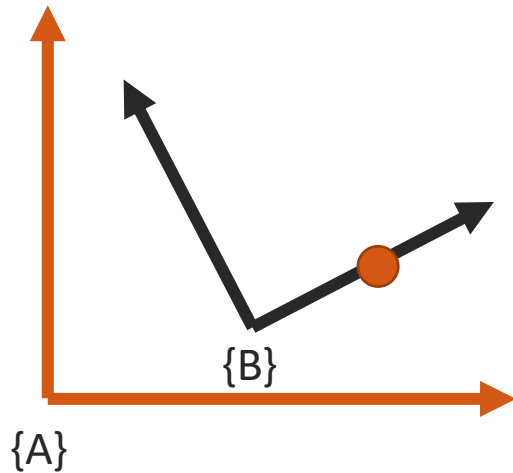
Starting with one axis:

How to map ${}^B \begin{bmatrix} 1 \\ 0 \end{bmatrix}$ onto A ?

$\{B\}_x$ contributes to both $\{A\}_x$ and $\{A\}_y$!

Nested Coordinate Systems: Projection

- **Problem:** How to map ${}^B \begin{bmatrix} 1 \\ 0 \end{bmatrix}$ onto A? $\{B\}_x$ contributes to both $\{A\}_x$ and $\{A\}_y$
- **Solution:** Need to find the amount of these contributions!



Recall:

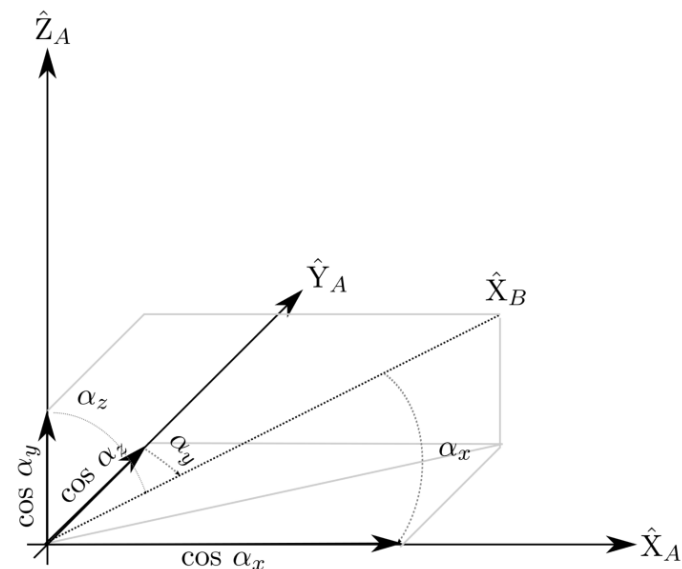
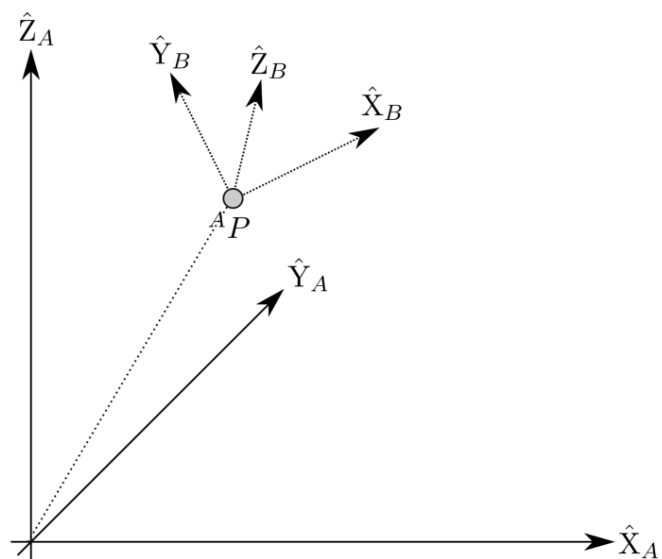
- Axes are unit-length vectors
- Dot product formula
 - $A \cdot B = \sum_i A_i B_i = |A| * |B| * \cos(\alpha)$

$\{B\}_x \cdot \{A\}_x = \text{Amount of } \{B\}_x \text{ that contributes to } \{A\}_x$

$\{B\}_x \cdot \{A\}_y = \text{Amount of } \{B\}_x \text{ that contributes to } \{A\}_y$

$${}^A_B R = \begin{bmatrix} \{B\}_x \cdot \{A\}_x & \{B\}_y \cdot \{A\}_x \\ \{B\}_x \cdot \{A\}_y & \{B\}_y \cdot \{A\}_y \end{bmatrix}$$

Expressing $\{B\}$ in $\{A\}$



$${}^A\hat{X}_B = (\hat{X}_B \cdot \hat{X}_A, \hat{X}_B \cdot \hat{Y}_A, \hat{X}_B \cdot \hat{Z}_A)^T$$

$${}^A\hat{Y}_B = (\hat{Y}_B \cdot \hat{X}_A, \hat{Y}_B \cdot \hat{Y}_A, \hat{Y}_B \cdot \hat{Z}_A)^T$$

$${}^A\hat{Z}_B = (\hat{Z}_B \cdot \hat{X}_A, \hat{Z}_B \cdot \hat{Y}_A, \hat{Z}_B \cdot \hat{Z}_A)^T$$

$${}^A_B R = [{}^A X_B, {}^A Y_B, {}^A Z_B] = \begin{bmatrix} \hat{X}_B \cdot \hat{X}_A & \hat{Y}_B \cdot \hat{X}_A & \hat{Z}_B \cdot \hat{X}_A \\ \hat{X}_B \cdot \hat{Y}_A & \hat{Y}_B \cdot \hat{Y}_A & \hat{Z}_B \cdot \hat{Y}_A \\ \hat{X}_B \cdot \hat{Z}_A & \hat{Y}_B \cdot \hat{Z}_A & \hat{Z}_B \cdot \hat{Z}_A \end{bmatrix}$$

Nested Coordinate Systems

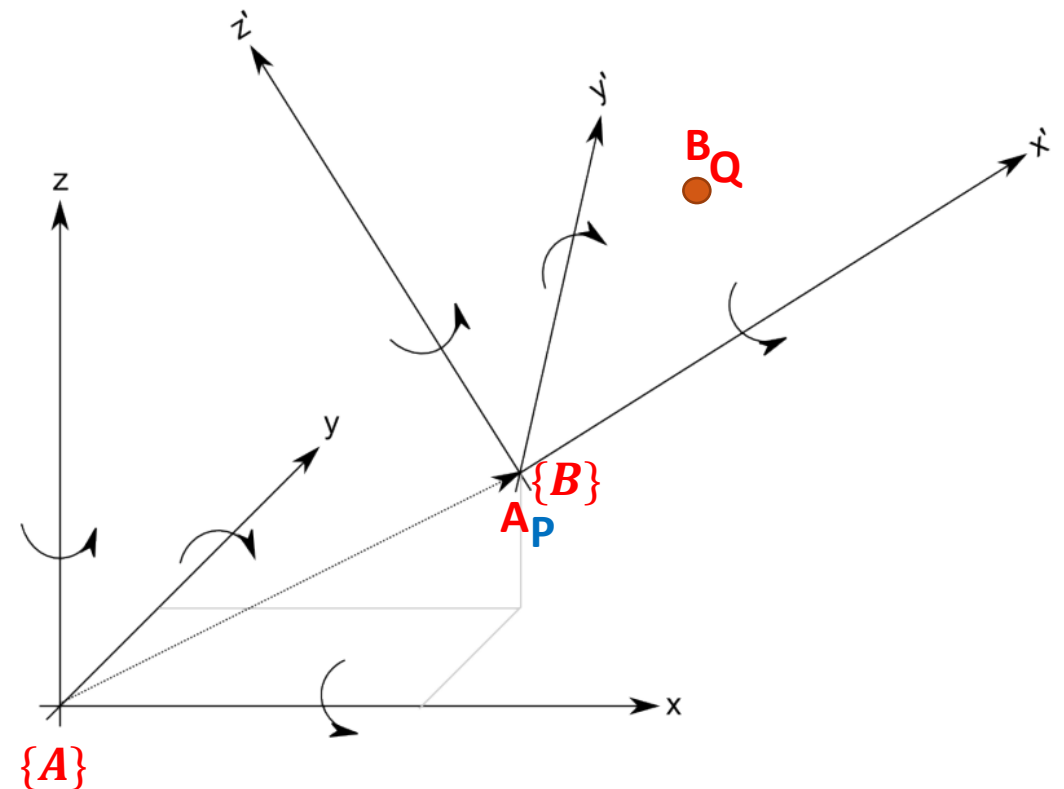
How can we express point Q , with position known in coordinate system

$\{B\}$ centered at P , in coordinate system

$\{A\}$ centered at the origin $(0,0)$?

$${}^B Q = \text{Known}$$

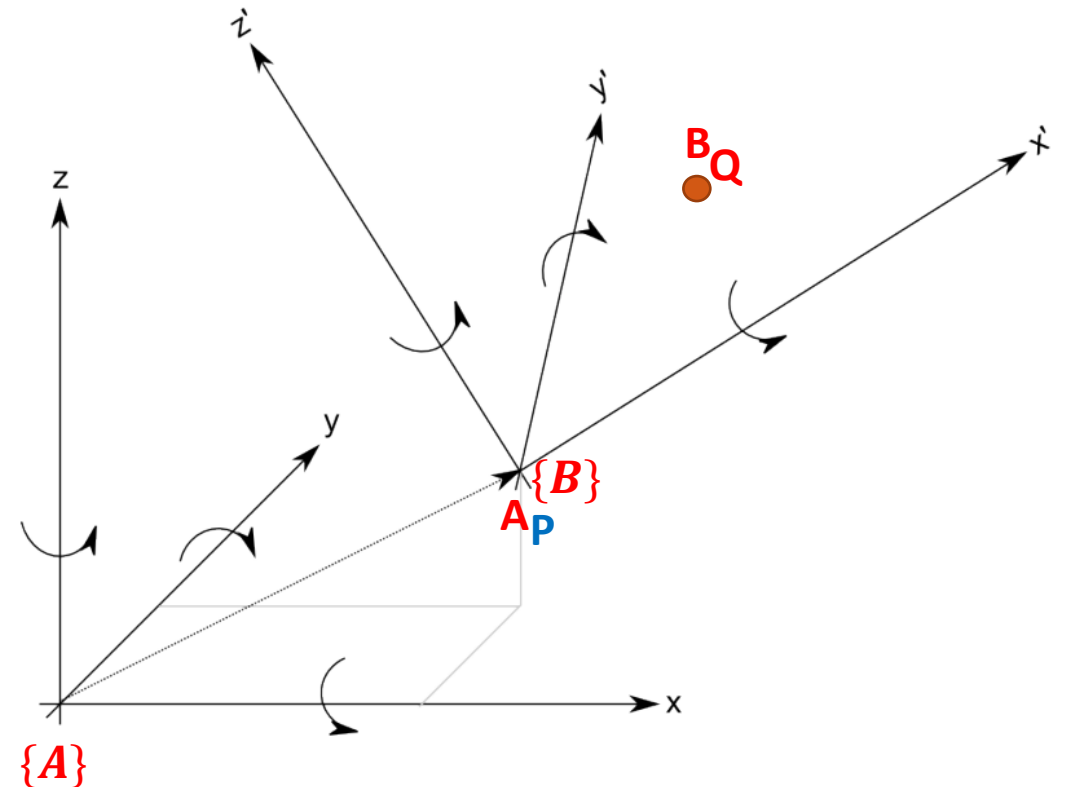
$${}^A P = \text{Known}$$



Nested Coordinate Systems

How can we express point Q , with position known in coordinate system $\{B\}$ centered at P , in coordinate system $\{A\}$ centered at the origin $(0,0)$?

- ${}^A_B R = [{}^A X_B, {}^A Y_B, {}^A Z_B]$
- ${}^A X_B = [X_B \cdot X_A, X_B \cdot Y_A, X_B \cdot Z_A]^T$
- ${}^A Q = {}^A_B R * {}^B Q + {}^A P$



Homogenous Transform

- Instead of ${}^A Q = {}^A_B R * {}^B Q + {}^A P$, we can express the transformation as a **single matrix multiplication**:

$$\begin{bmatrix} {}^A Q \\ 1 \end{bmatrix} = \begin{bmatrix} {}^A_B R & {}^A P \\ 0 & 1 \end{bmatrix} \begin{bmatrix} {}^B Q \\ 1 \end{bmatrix}$$

- Inverse Transform:

$$T^{-1} = \begin{bmatrix} R^T & -R^T {}^A P \\ 0 & 1 \end{bmatrix} \in SE(3)$$

Why Matrices?

Two primary uses:

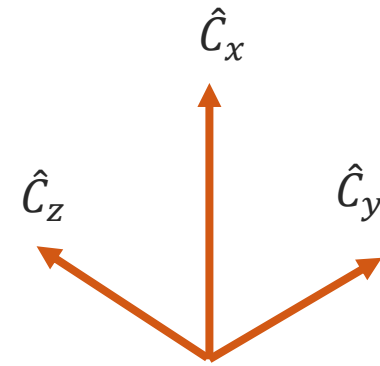
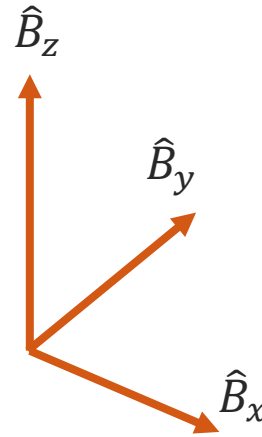
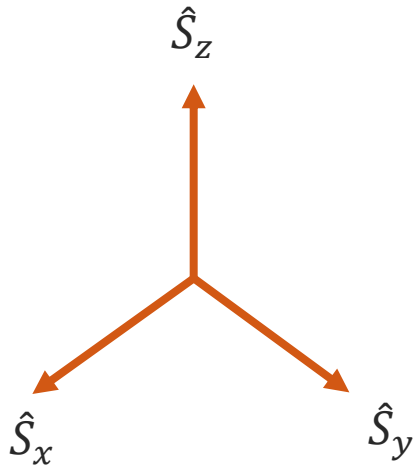
- Rotations and **transformations** of objects
 - Rotate the robot in place
 - Move the robot along a vector
- **Expressing coordinate system changes**
 - Given the location of an object in the robot's coordinate frame, express the object in the world coordinate frame
(e.g., the middle of the start line on the line following map)

Example

$${}^A\hat{X}_B = (\hat{X}_B \cdot \hat{X}_A, \hat{X}_B \cdot \hat{Y}_A, \hat{X}_B \cdot \hat{Z}_A)^T$$

$${}^A\hat{Y}_B = (\hat{Y}_B \cdot \hat{X}_A, \hat{Y}_B \cdot \hat{Y}_A, \hat{Y}_B \cdot \hat{Z}_A)^T$$

$${}^A\hat{Z}_B = (\hat{Z}_B \cdot \hat{X}_A, \hat{Z}_B \cdot \hat{Y}_A, \hat{Z}_B \cdot \hat{Z}_A)^T$$



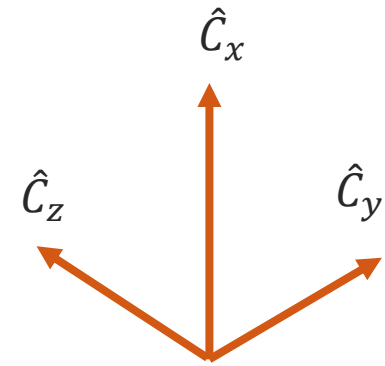
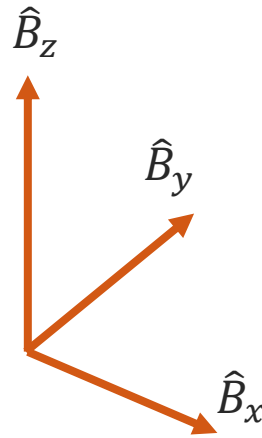
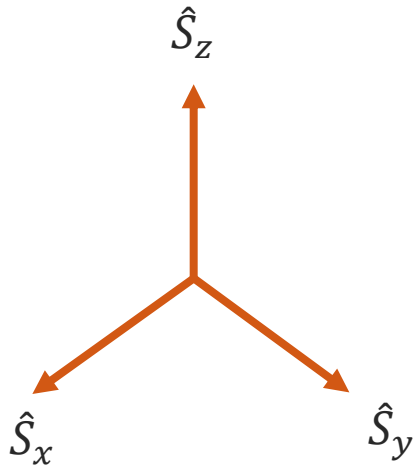
$${}_{\hat{C}}^{\hat{S}}R = \begin{bmatrix} 0 & -1 & 0 \\ 0 & 0 & -1 \\ 1 & 0 & 0 \end{bmatrix}$$

X-axis of $\hat{S}$ is the negative-Y-axis of $\hat{C}$

Y-axis of $\hat{S}$ is the negative-Z-axis of $\hat{C}$

Z-axis of $\hat{S}$ is the X-axis of $\hat{C}$

Example: Expressing a Point in New Coordinate Frame

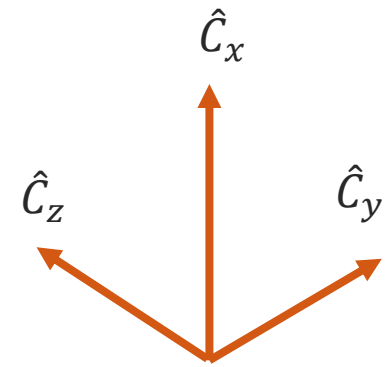
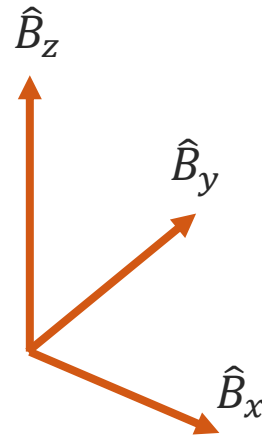
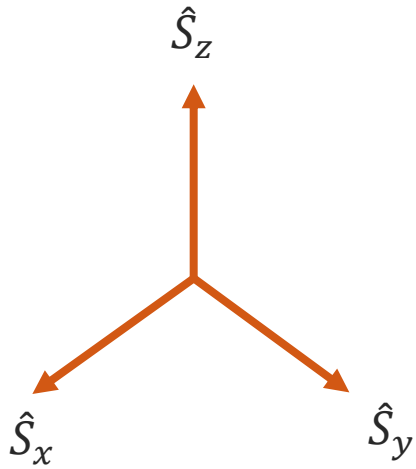


$${}^{\hat{S}}_{\hat{C}}R = \begin{bmatrix} 0 & -1 & 0 \\ 0 & 0 & -1 \\ 1 & 0 & 0 \end{bmatrix}$$

$${}^{\hat{C}}p = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}$$

$${}^{\hat{S}}p = {}^{\hat{S}}_{\hat{C}}R {}^{\hat{C}}p = \begin{bmatrix} 0 & -1 & 0 \\ 0 & 0 & -1 \\ 1 & 0 & 0 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}$$

Example: Inverting a Rotation

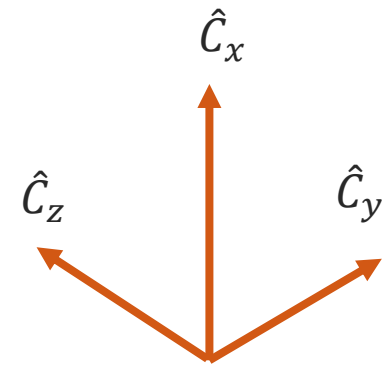
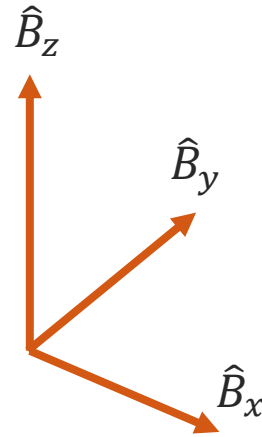
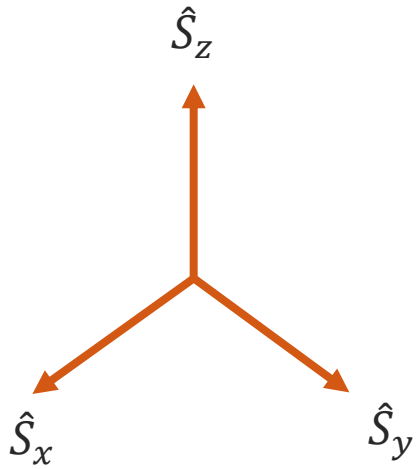


$$\hat{S}_{\hat{C}}R = \begin{bmatrix} 0 & -1 & 0 \\ 0 & 0 & -1 \\ 1 & 0 & 0 \end{bmatrix}$$

$$\hat{C}_{\hat{S}}R = \begin{bmatrix} 0 & 0 & 1 \\ -1 & 0 & 0 \\ 0 & -1 & 0 \end{bmatrix}$$

Transpose!

Example: Composing Rotations

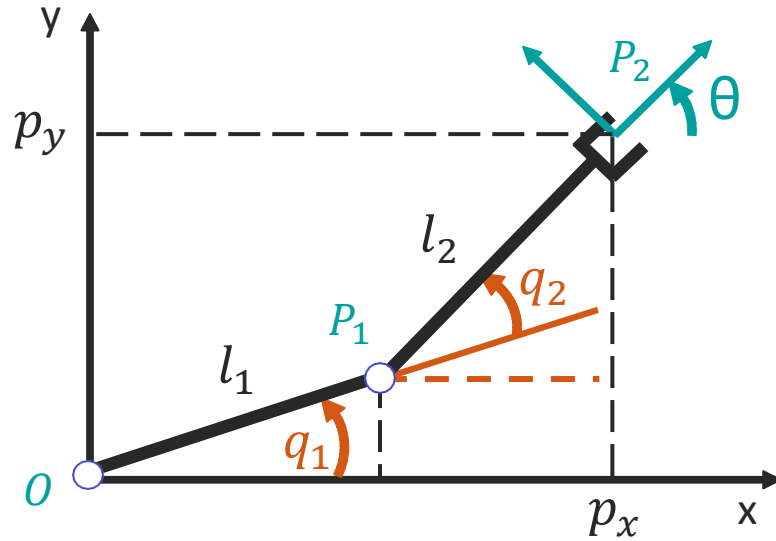


$$\hat{S}_{\hat{B}}R = \begin{bmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

$$\hat{B}_{\hat{C}}R = \begin{bmatrix} 0 & 0 & -1 \\ 0 & 1 & 0 \\ 1 & 0 & 0 \end{bmatrix}$$

$$\hat{S}_{\hat{C}}R = \hat{S}_{\hat{B}}R \hat{B}_{\hat{C}}R = \begin{bmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 0 & 0 & -1 \\ 0 & 1 & 0 \\ 1 & 0 & 0 \end{bmatrix} = \begin{bmatrix} 0 & -1 & 0 \\ 0 & 0 & -1 \\ 1 & 0 & 0 \end{bmatrix}$$

Example: direct kinematics of 2R arm



Forward Kinematics of a differential wheel robot (i.e. the e-puck)

- **Manipulator:**

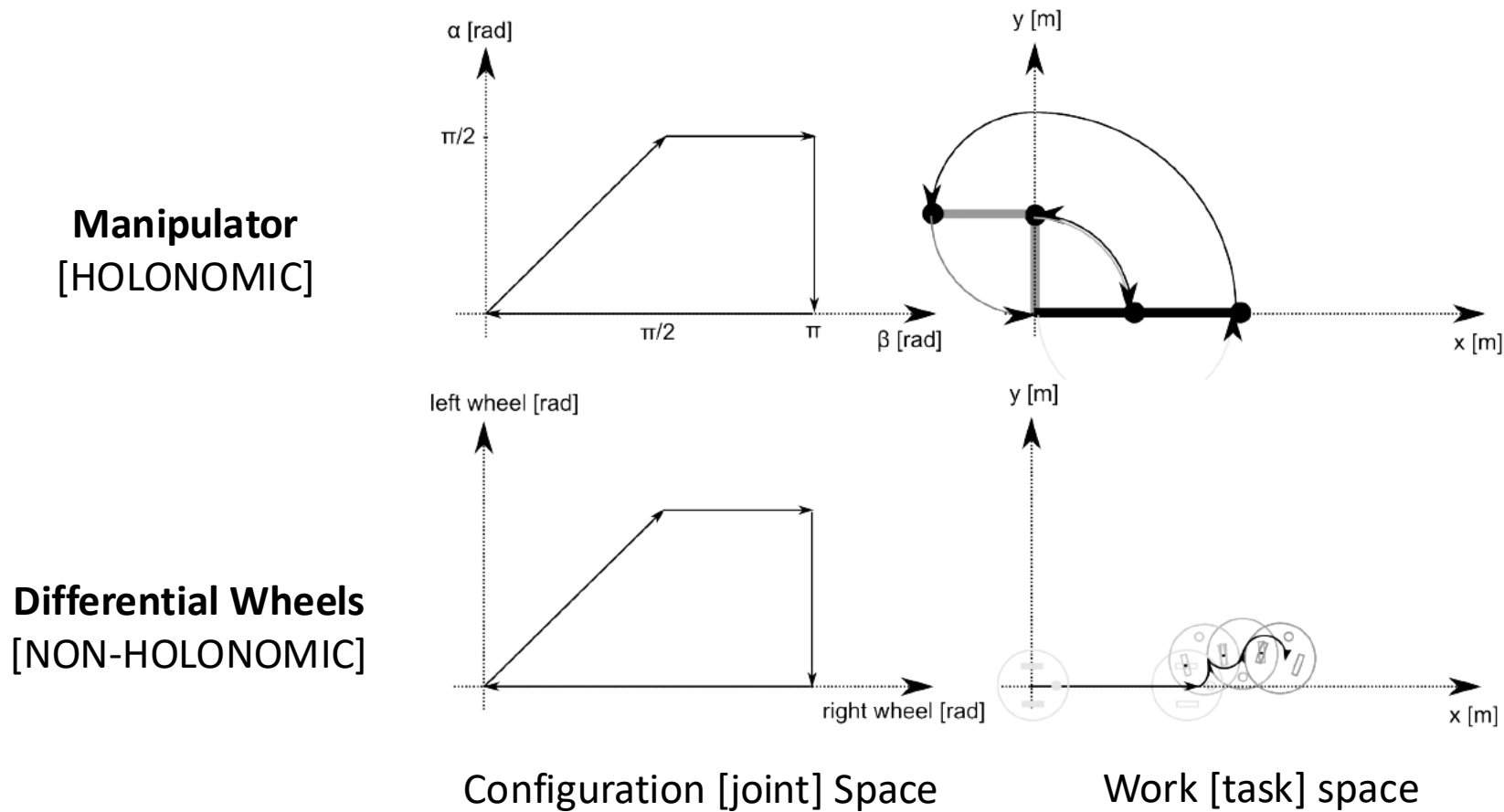
- Forward kinematics is uniquely defined by its joint angles
- Measure of joint angles through encoders is **absolute**

- **Differential Wheels** robot:

- Measure of joint angles through encoders is **relative**
- Encoders' values need to be integrated **over time**

Please take a look at Section 3.3.2 in the book

Forward Kinematics of a differential wheel robot (i.e. the e-puck)



Please take a look at Section 3.3.2 in the book

Holonomic or Non-Holonomic on the 2D plane?

- Steering wheel is rotated 90 degrees then acceleration is applied for 1 second

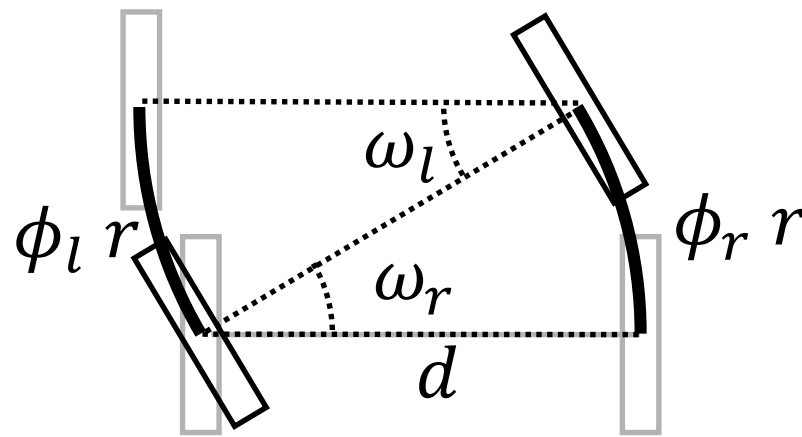
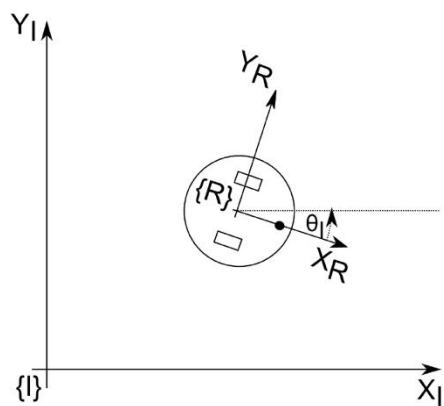
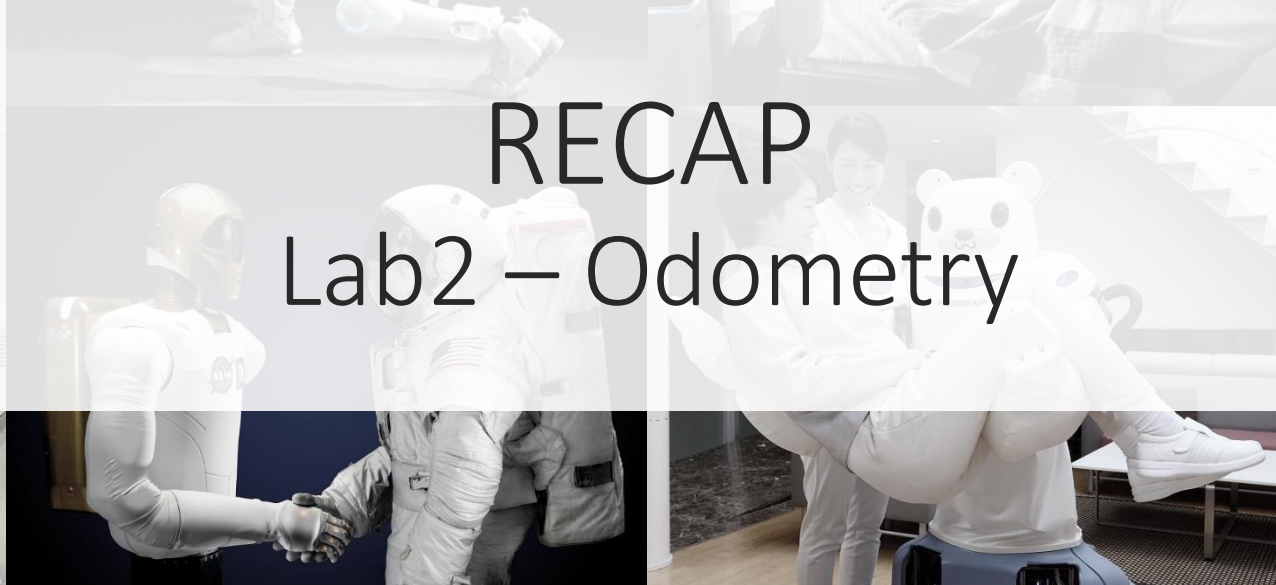
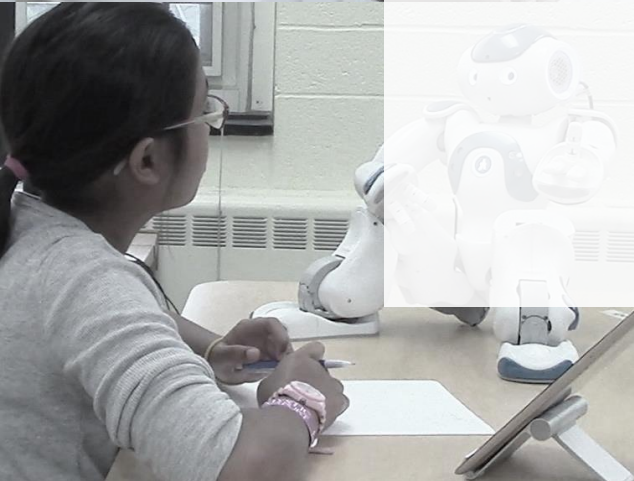
vs.

- Acceleration is applied for 1 second then steering wheel rotated 90 degrees!
- Different ending configuration = **Non-holonomic!**



Chapter 3

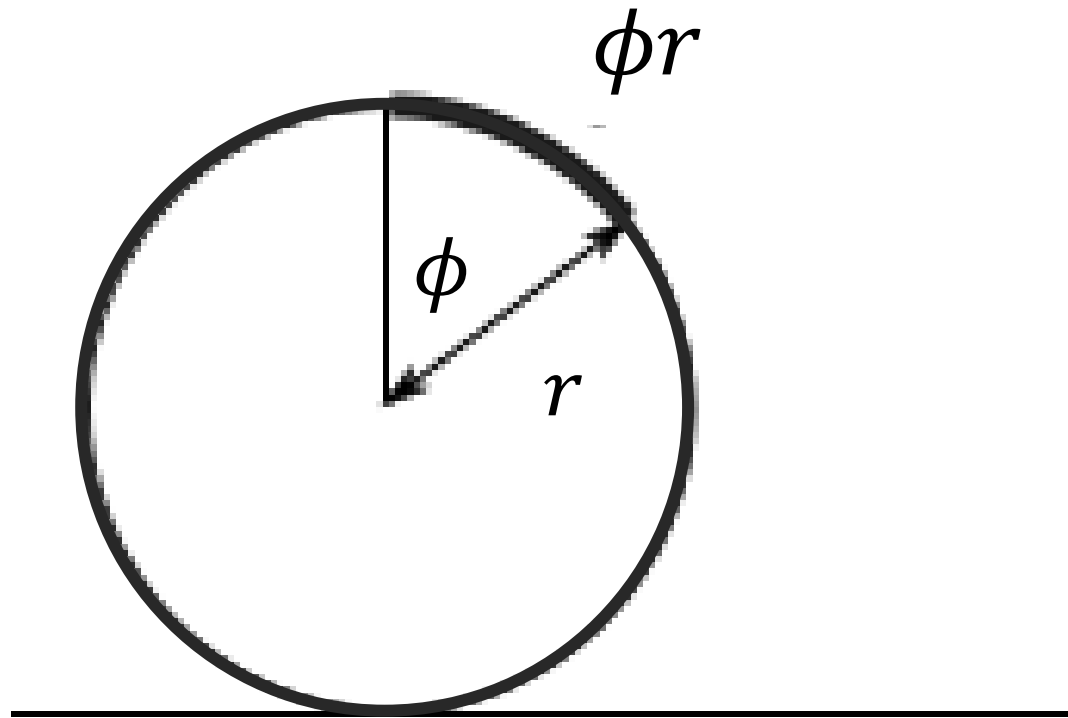
RECAP Lab2 – Odometry



Symbols

- $\phi \rightarrow$ “phi” $\leftarrow$ angular displacement of single wheel
- $\omega \rightarrow$ “omega” $\leftarrow$ angular displacement caused by each wheel
- $\theta \rightarrow$ “theta” $\leftarrow$ orientation of robot
- $\dot{} \rightarrow$ time derivative (e.g. $\dot{\phi}_l \rightarrow$ derivative (rate of change) of “phi” over time)

1. Wheel motion

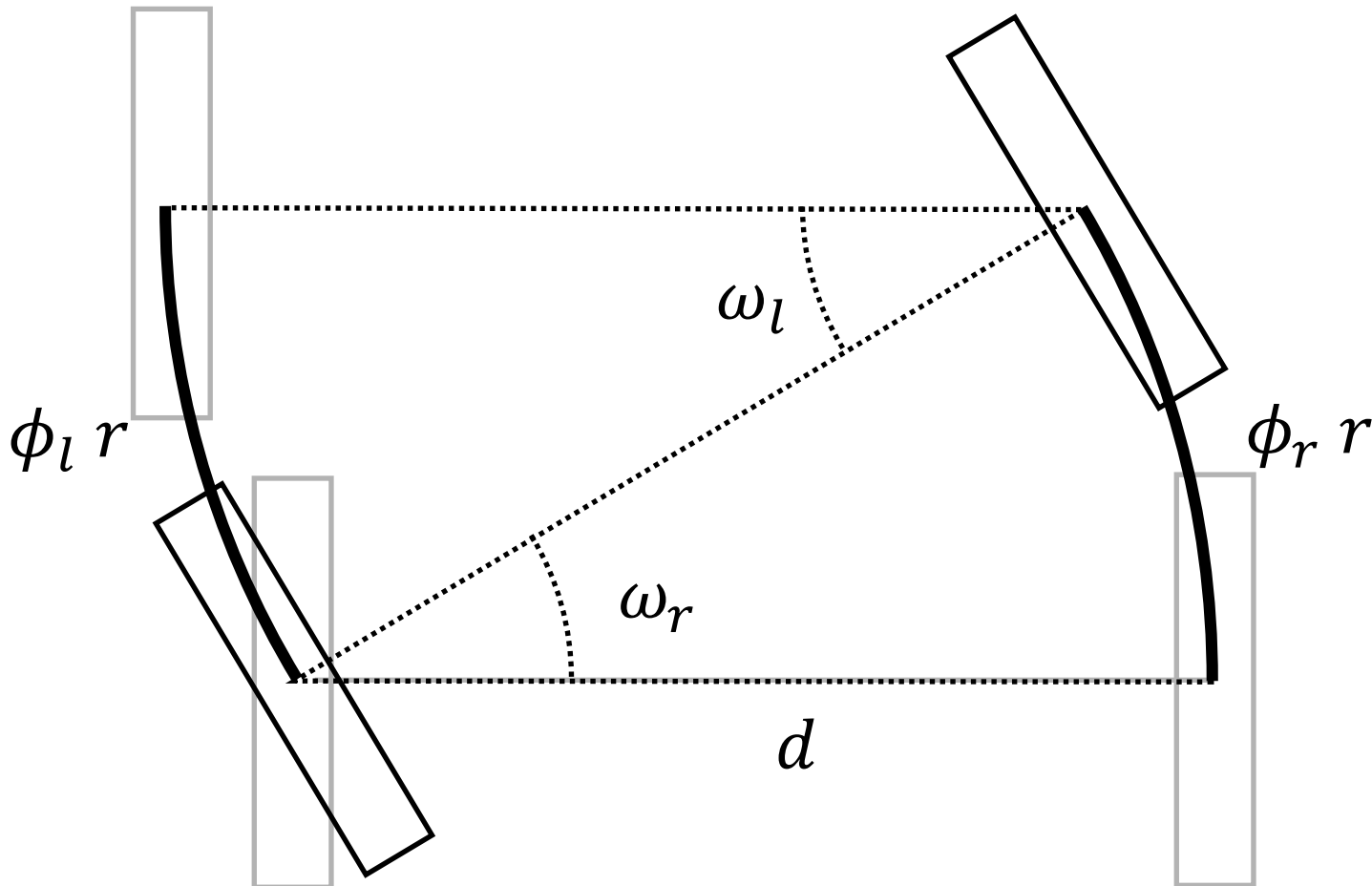


Distance traveled is angle of rotation times wheel radius:

$$x = r\phi$$

$$\dot{x} = r\dot{\phi}$$

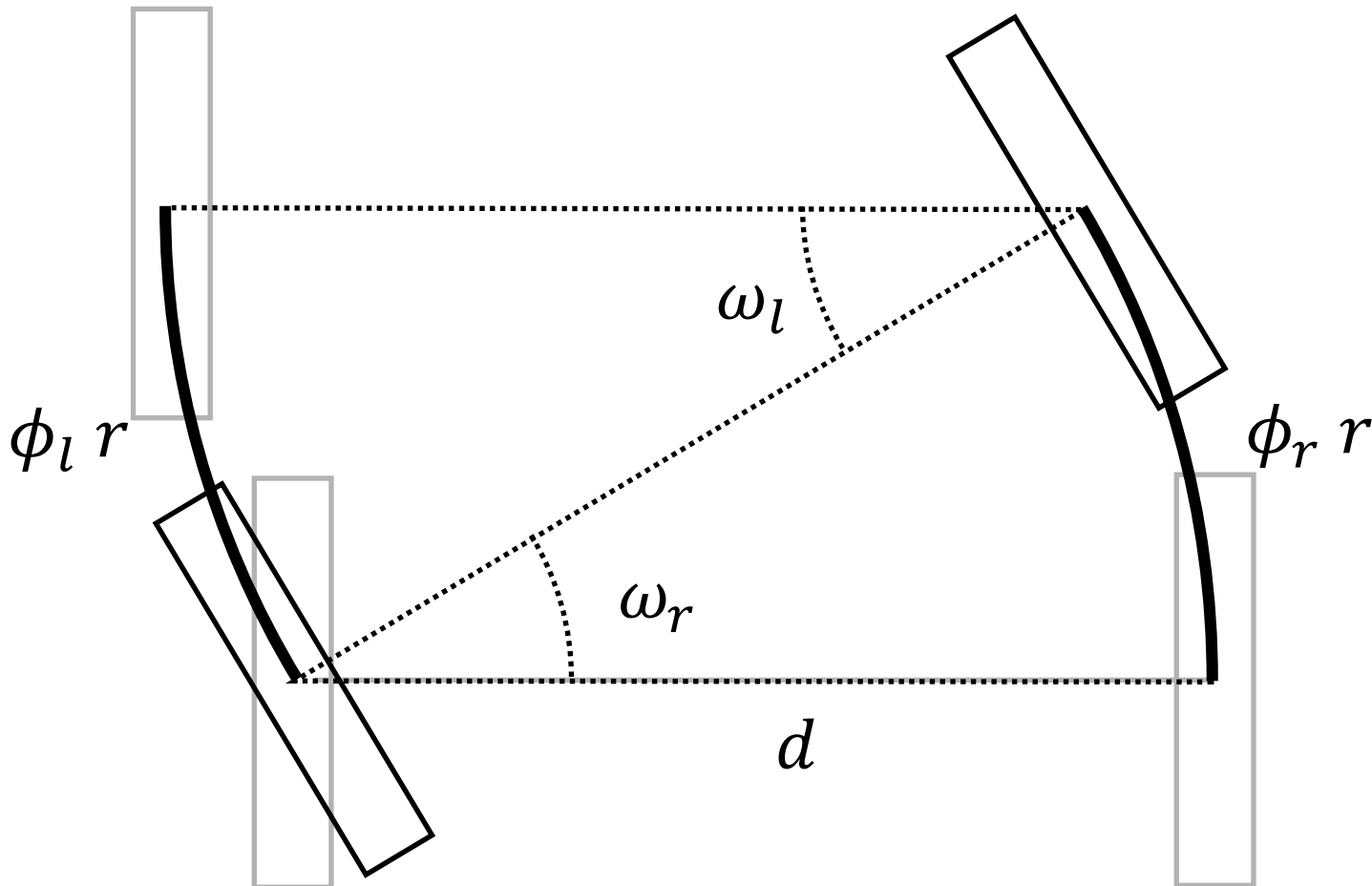
2. Wheel motions $\rightarrow$ Position Updates



What about the case where **both wheels** are moving at different speeds $\dot{\phi}_l$ and $\dot{\phi}_r$?

$$\dot{x}_r = \frac{r\dot{\phi}_l}{2} + \frac{r\dot{\phi}_r}{2}$$

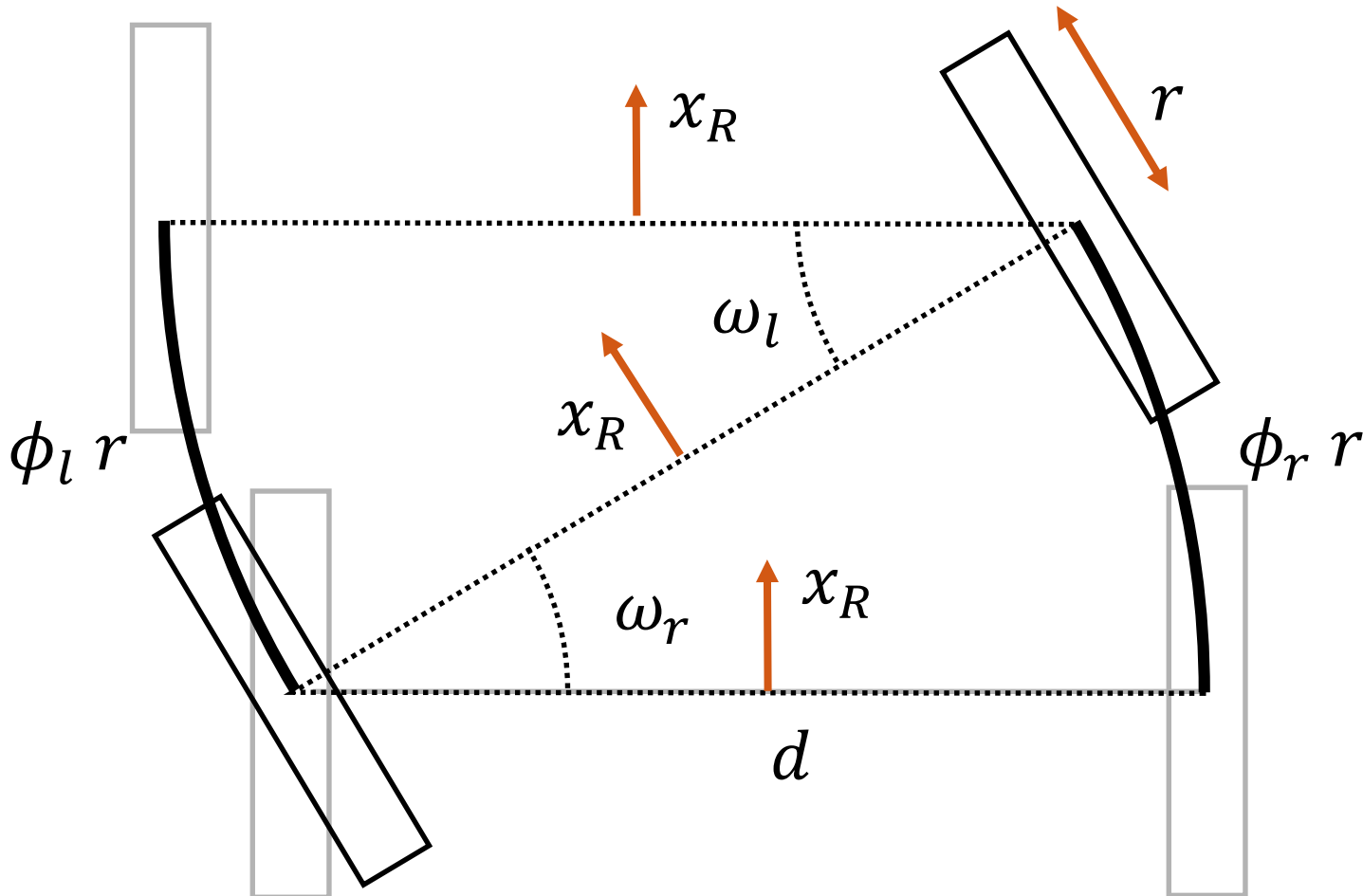
2. Wheel motions $\rightarrow$ Position Updates



$$\dot{\omega}_r = \frac{\dot{\phi}_r r}{d}$$
$$\dot{\omega}_l = \frac{\dot{\phi}_l r}{d}$$

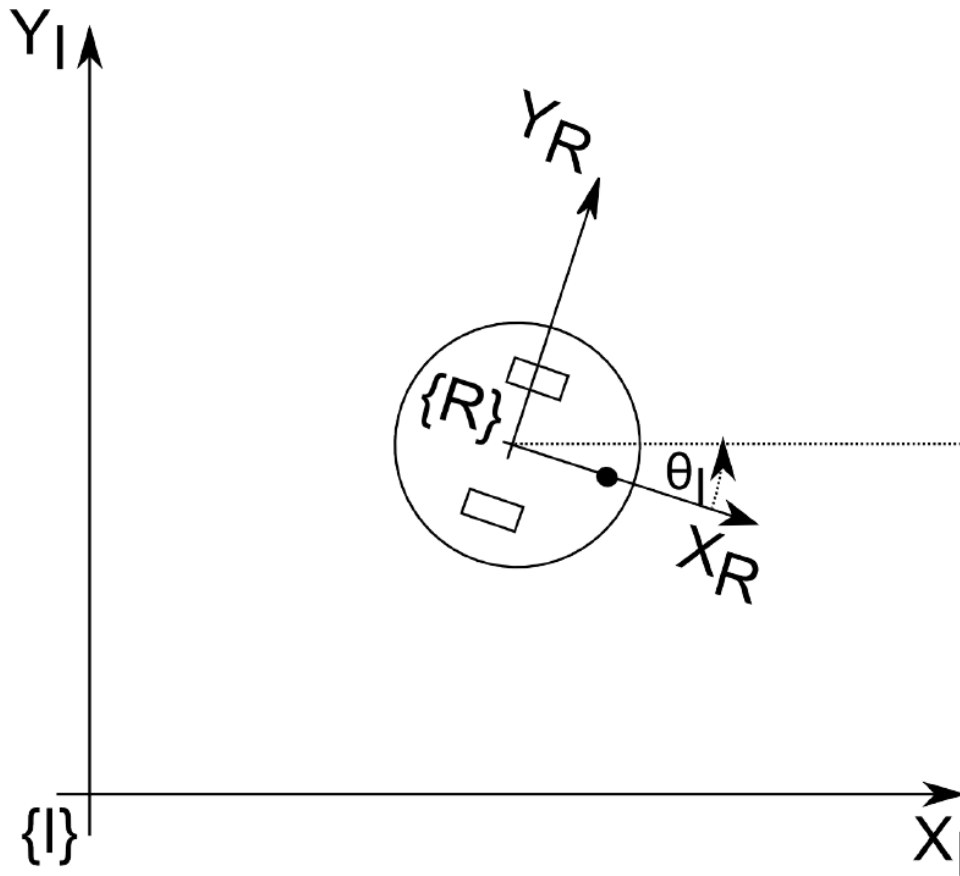
$$\dot{\theta} = \frac{\dot{\phi}_r r}{d} - \frac{\dot{\phi}_l r}{d}$$

2. Forward Kinematics of mobile robot



$$\begin{bmatrix} \dot{x}_R \\ \dot{y}_R \\ \dot{\theta}_R \end{bmatrix} = \begin{bmatrix} \frac{r\dot{\phi}_l}{2} + \frac{r\dot{\phi}_r}{2} \\ 0 \\ \frac{\dot{\phi}_r r}{d} - \frac{\dot{\phi}_l r}{d} \end{bmatrix}$$

3. Forward Kinematics + Odometry



$$\dot{x}_{I,x} = \cos(\theta) \dot{x}_R$$

$$\dot{x}_{I,y} = -\sin(\theta) \dot{y}_R$$

$$\dot{x}_I = \cos(\theta) \dot{x}_R - \sin(\theta) \dot{y}_R$$

$$\dot{y}_I = \sin(\theta) \dot{x}_R + \cos(\theta) \dot{y}_R$$

$$\dot{\theta}_I = \dot{\theta}_R$$

$$\begin{pmatrix} \dot{x}_I \\ \dot{y}_I \\ \dot{\theta} \end{pmatrix} = \begin{pmatrix} \cos(\theta) & -\sin(\theta) & 0 \\ \sin(\theta) & \cos(\theta) & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} \frac{r\dot{\phi}_l}{2} + \frac{r\dot{\phi}_r}{2} \\ 0 \\ \frac{\dot{\phi}_r r}{d} - \frac{\dot{\phi}_l r}{d} \end{pmatrix}$$

4. Speeds → How can we compute positions?

$$\begin{pmatrix} x_I(T) \\ y_I(T) \\ \theta(T) \end{pmatrix} = \int_0^T \begin{pmatrix} \dot{x}_I(t) \\ \dot{y}_I(t) \\ \dot{\theta}(t) \end{pmatrix} dt \approx \sum_{k=0}^{k=T} \begin{pmatrix} \Delta x_I(k) \\ \Delta y_I(k) \\ \Delta \theta(k) \end{pmatrix} \Delta t$$